

What Is a Pull Request, and How Does It Work Here?

A plain-language guide, plus the specific process for the nextvillage / AI'ing & Vibing platform

Written July 2026, after branch protection and CI checks were put in place for main.

1. What Is a Pull Request?

A **Pull Request** (PR) is a request to merge a set of code changes from one branch into another — almost always into the main branch that represents what's actually live in production. It's called a "pull request" because, historically, you're asking the project's maintainer to *pull* your changes into the official codebase.

In practice, a PR is a page on GitHub that shows:

- Every line of code that would change if the PR were merged
- Who wrote it and when
- Any comments, questions, or requested changes from reviewers
- Whether automated checks (tests, builds) passed or failed
- A single button to merge it in, once everything looks good

The key idea: instead of directly editing the live codebase, you propose a change, let it be checked (by a person, a machine, or both), and only then does it become part of the real thing.

2. Why Use Pull Requests at All?

Without this process, anyone with access can push a change straight to the live codebase, and it takes effect immediately — no check, no second opinion, no undo button beyond digging through history. That's exactly what happened here in June 2026: a change to an unrelated file accidentally disabled the platform's profile-completion popup for six weeks before anyone noticed, because nothing verified the change before it went live.

A PR process fixes this by inserting a checkpoint between "I wrote some code" and "this is now live for every user." Three concrete benefits:

- **Automated safety net.** A computer checks the change automatically — does it still compile? Does it still build? Do the tests still pass? — every single time, with no risk of forgetting to run the check by hand.
- **A record of what changed and why.** Every PR has a title, a description, and a permanent link to the exact diff. Six months from now, "why did we do this?" has an answer.
- **A natural place for a second opinion.** Even without requiring a formal review, a PR is somewhere a teammate *can* look before it merges, instead of finding out after the fact that

something broke.

3. A Few Terms Worth Knowing

Repository (repo)

The project's full codebase and history, stored in git and hosted on GitHub.

Branch

An independent line of work. You make your changes on your own branch so they don't affect anyone else's work — or the live site — until they're merged.

Commit

A saved snapshot of changes, with a message describing what changed. A PR is usually made up of one or more commits.

Push

Uploading your local commits to GitHub, so others (and GitHub itself) can see them.

CI (Continuous Integration)

The automated process that runs every time code is pushed or a PR is opened — in this project, that means checking the code compiles, the test suite passes, and the production build succeeds.

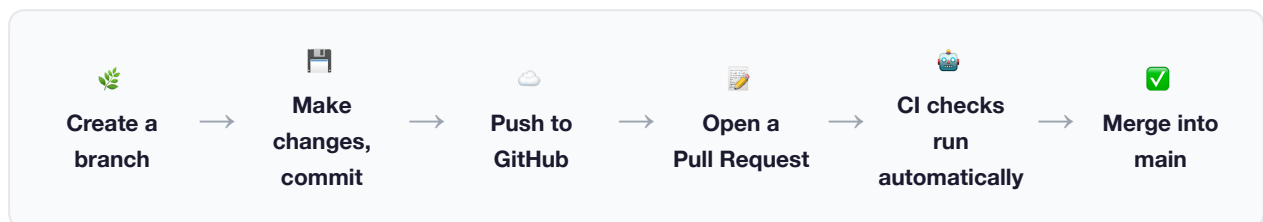
Merge

Combining a branch's changes into another branch — in our case, taking an approved PR's changes and folding them into `main`.

Branch protection

A GitHub setting on a specific branch (here, `main`) that enforces rules — e.g. "you can't push directly, you must open a PR, and it must pass CI first."

4. The Process, Visually



5. What's Specifically Configured for This Platform

Rule	What it means in practice
Direct pushes to <code>main</code> are blocked	You cannot type <code>git push origin main</code> and have it work — GitHub will reject it, for everyone except the repo owner (Kevin).

A Pull Request is required to merge into <code>main</code>	All changes, no matter how small, go through the PR page on GitHub, not a direct push.
The <code>typecheck-and-build</code> check must pass	Every PR automatically runs three steps: TypeScript typechecking, the Vitest test suite, and a full production build. If any step fails, the PR shows a red ✗ and cannot be merged until it's fixed.
No required human approval count (currently 0)	A PR does not need someone else to click "Approve" before it can merge — kept deliberately lightweight for a small team. This can be tightened later if needed.
Admin bypass exists, but shouldn't be the default	Kevin's account can technically push directly to <code>main</code> in a genuine emergency. Even Kevin should default to using a PR like everyone else.

6. Step-by-Step: Making a Change

1 Start from an up-to-date branch

```
git checkout development
git pull origin development
git checkout -b your-name/short-description
```

Example: `git checkout -b divinegift/fix-typo-homepage`

2 Make your changes, then commit them

```
git add .
git commit -m "Describe what changed and why"
```

3 Push your branch to GitHub

```
git push -u origin your-name/short-description
```

4 Open the Pull Request

GitHub shows a banner with a "Compare & pull request" button right after you push — click it. Or go to the repository on GitHub.com → **Pull requests** → **New pull request**. Set the base branch to `main` and your branch as the one being compared.

5 Wait for the automatic check, then merge

Within about a minute, the PR page shows either a green checkmark or a red X next to `typecheck-and-build`.

- **Green:** click **Merge pull request**. Done — your change is now live once Vercel finishes deploying.

- **Red:** click "Details" next to the failed check to read exactly what broke, fix it on the same branch, commit, and push again — the check re-runs automatically on the new commit.

7. Frequently Asked

Do I need someone to approve my PR before merging?

Not currently — only the automated check needs to pass. You can merge your own PR once it's green.

What if I'm not sure my change is even correct?

Open the PR anyway. It's a safe place to get a second pair of eyes — ask a teammate to look before you click merge, even though it's not required.

What does the check actually verify?

That the code compiles without type errors, the automated test suite passes, and the app successfully builds for production. It does *not* verify that a feature works correctly in the browser — that's still on the person making the change to check by hand for anything user-facing.

I don't have a GitHub account yet — what do I do?

Create one at github.com, then send your username to Kevin so he can add you as a collaborator on the repository. Until then, you won't be able to push branches or open PRs against this repo.

Companion document to [docs/GitHub-Push-Process-Report.pdf](#) in the repository, which is version-controlled alongside the code. This copy lives outside the repo for easy sharing/reference.